

Google Summer of Code 2026 Proposal

Post-Solver Feasibility and Optimality Checks for CVXPY

Contributor: Suleyman Talha Belen
Email: talha.belen@ug.bilkent.edu.tr
GitHub: <https://github.com/stalhabelen>
University: Bilkent University, Dept. of Electrical & Electronics Engineering
Year: 3rd year (Junior)
Organization: NumFOCUS / CVXPY
Project Size: 350 hours (Large)
Mentors: William Zhang, Daniel Cederberg, Parth Nobel

Abstract

CVXPY is a widely-used Python-embedded modeling language for convex optimization. Users write problems in a natural mathematical form; CVXPY canonicalizes them into a standard form and delegates the actual solve to a third-party solver (Clarabel, SCS, MOSEK, etc.). A fundamental trust assumption in this pipeline is that the solver's reported status—"optimal", "infeasible", etc.—faithfully reflects the quality of the solution in the user's original problem space. In practice, the canonicalization transformations can amplify numerical error, so a solution that is accurate in the solver's transformed space may violate the user's original constraints by orders of magnitude.

This project adds a **post-solver verification layer** to CVXPY that independently checks:

1. **Feasibility** — whether the returned primal values satisfy every constraint of the *original* problem to a user-configurable tolerance, and
2. **Optimality** — whether the KKT (Karush–Kuhn–Tucker) conditions hold for the returned primal–dual pair, extending the prototype Lagrangian class from GSoC 2023 and extending support to DNLP (Disciplined Nonlinear Programming) problems solved via IPOPT.

Background & Motivation

The Problem

When a user calls `problem.solve()`, CVXPY performs a multi-step canonicalization:

User problem $\xrightarrow{\text{rewrite}}$ Cone program (standard form) $\xrightarrow{\text{solver}}$ Solution $\xrightarrow{\text{inverse map}}$ User-space values

At each arrow, numerical precision can be lost. The solver may report **optimal** within its own tolerance (e.g. 10^{-6}), yet after the inverse mapping the user-space constraint violation can be $O(1)$ or even $O(10)$ [1].

Prior Work — GSoC 2022 & 2023

The need for post-solver verification in CVXPY was first raised in 2018 with Issue #434 [1], which observed that a solver may report "optimal" within its own internal tolerance, yet after CVXPY's inverse mapping the user-space constraint violation can be orders of magnitude larger. The proposed safeguard—a configurable feasibility override tolerance that checks every constraint before returning "optimal"—remains unimplemented as of 2026.

Aryaman Jeendgar — GSoC 2022 & 2023. In GSoC 2022, Jeendgar contributed to CVXPY’s conic modelling infrastructure [3], developing how the canonicalization pipeline transforms user-level constraints into solver-level cone programs and how dual variables are mapped back. Building on this experience, in GSoC 2023 he built a prototype **Lagrangian** class that can: (a) automatically construct the Lagrangian $\mathcal{L}(x, \lambda) = f(x) + \sum_i \lambda_i g_i(x)$ from a **Problem** object; (b) compute $\nabla_x \mathcal{L}$ using CVXPY’s atom-level derivative information and the chain rule; and (c) evaluate all four KKT conditions (stationarity, primal feasibility, dual feasibility, complementary slackness) at a given primal–dual pair. He also implemented a pure NumPy canonicalization backend that provides a clean dense-array reference implementation useful for correctness verification.

What Remains Incomplete. Despite this progress, none of Jeendgar’s optimality code has been merged into CVXPY’s main branch. The specific gaps are:

1. **No public API integration.** The Lagrangian class exists only in a separate branch/fork. It is not importable from the `cvxpy` package and is not triggered by `problem.solve()`. In practice, no CVXPY user currently benefits from it.
2. **DNLP support is entirely absent.** When CVXPY solves a DNLP problem via IPOPT, the solver returns Lagrange multipliers for its internal NLP representation, but the inverse dual mapping was never implemented. `constraint.dual_value` is always `None` for DNLP problems, making KKT verification impossible.
3. **Incomplete test coverage.** The prototype was tested on a narrow set of problem types. No systematic tests exist for different cone types (SOCP, SDP, exponential, power), DNLP-specific atoms, or edge cases like complex variables, scaling issues, or mixed equality/inequality constraints.
4. **Feasibility checks (Issue #434) were not addressed.** The GSoC 2023 project focused solely on optimality. The feasibility verification safeguard proposed in 2018 still does not exist.
5. **No documentation.** There is no developer or user-facing documentation for the Lagrangian class or its interaction with CVXPY’s internals.

This project picks up exactly where GSoC 2023 left off: it will integrate and extend the Lagrangian prototype, build the feasibility verification layer that Issue #434 has been requesting since 2018, add DNLP dual-variable support, and provide the comprehensive test suite and documentation needed to make these features production-ready.

Why This Matters

CVXPY is used by a broad audience—scientists, engineers, students—many of whom reasonably trust a status of "optimal". A post-solver verification layer provides a critical safety net, turning CVXPY from “trust the solver” to “trust but verify.”

Technical Approach

I divide the project into two largely independent tracks that can proceed in parallel once the initial codebase review is complete.

Track A: Post-Solver Feasibility Checks

Proposed Design. After the solver returns, iterate over every **Constraint** object in the original **Problem** and compute its violation using the existing `constraint.violation()` API. Compare the maximum violation against a configurable tolerance τ (default 10^{-4} , following the suggestion in Issue #434).

```

max_viol = max(c.violation() for c in problem.constraints)
if max_viol > tau:
    warnings.warn(
        f"Post-solver feasibility check failed: "
        f"max constraint violation = {max_viol:.2e} "
        f"exceeds tolerance {tau:.2e}. "
        f"Status overridden to POST_SOLVER_INFEASIBLE."
    )
    problem._status = POSTCHECKS_INFEASIBLE

```

Solver-specific tolerance handling. Different solvers expose different tolerance parameters (`eps_abs` for SCS, `MSK_DPAR_INTPNT.TOL_PFEAS` for MOSEK, etc.). The feasibility checker will read the solver-specific tolerance and use $\tau = \max(\text{user_tol}, k \times \text{solver_tol})$ where k is a configurable scaling factor, ensuring the check accounts for solver precision. I will implement and test tolerance extraction for each major solver: SCS (`eps_abs`, `eps_rel`), Clarabel (`tol_gap_abs`, `tol_feas`), MOSEK (`MSK_DPAR_INTPNT.TOL_PFEAS`), ECOS (`abstol`, `feastol`), OSQP (`eps_abs`, `eps_rel`), HiGHS (`primal_feasibility_tolerance`), and IPOPT (`tol`, `constr_viol_tol`).

Edge cases to test.

- **Empty constraint set.** If the problem has no constraints, the check should trivially pass without errors.
- **Mixed equality / inequality constraints.** The `violation()` computes residuals differently for equalities vs. inequalities (absolute deviation from zero vs. positive part). I will write tests that mix `Zero` and `NonPos` constraints and verify the max violation is computed correctly across both.
- **Very large / very small coefficients (scaling issues).** This is one of the most commonly reported problems in CVXPY (e.g., Issues #681, #261). When input data is scaled to $\sim 10^{-8}$ or smaller, solver stopping criteria can be fooled, and the returned solution may be garbage while still claiming “optimal.” I will construct LPs/QPs with known solutions, scale all data by 10^{-6} and 10^6 , solve with multiple solvers, and verify the feasibility checker catches violations that the solver missed.
- **Infeasible and unbounded problems.** When the solver returns `infeasible`, `unbounded`, or `infeasible_or_unbounded`, there is no solution to check, so the feasibility check should be skipped entirely. I will test clearly infeasible problems, clearly unbounded problems, and the combined status I will also test the `optimal_inaccurate` status, where a solution does exist but the tolerance comparison needs special care.
- **Complex-valued variables.** CVXPY supports `cp.Variable(complex=True)`, but the interaction with constraint violations is tricky (cf. Issues #907, #1816). I will ensure `violation()` returns a real, non-negative number even for complex expressions, and test with PSD constraints where CVXPY internally decomposes the complex constraint into a real symmetric one.
- **PSD (matrix) constraints.** The violation for $X \succeq 0$ is measured as the magnitude of the most negative eigenvalue. Several issues (e.g., #2362, #674) show these are fragile. I will test with known-solution SDPs across SCS, MOSEK, and Clarabel, including a larger matrix case (50×50) to test numerical precision.
- **All supported solvers.** Different solvers produce solutions of very different quality (e.g., Issues #1320, #922, #1267). For each solver, I will construct a poorly conditioned QP and verify that the feasibility checker correctly reads the solver’s tolerance, computes τ , and issues warnings or overrides when appropriate—without false alarms when the violation is within τ .

Track B: Post-Solver Optimality Checks

The goal of Track B is to give CVXPY users an independent way to verify that a solver’s returned primal–dual pair actually satisfies KKT conditions in the user’s original problem space. This covers both the DCP path and the DNLP path.

Step 1: Review and integrate the GSoC 2023 Lagrangian class. I studied the prototype and will produce a clean, tested version integrated into `cvxpy/verification/` (new subpackage). The public API will be a method on `Problem`:

```
1 result = problem.check_kkt(tol=1e-4)
2 # Returns: stationarity_viol, primal_infeas,
3 #          dual_infeas, comp_slack_viol, passed (bool)
```

This method internally constructs the Lagrangian, retrieves primal values from `variable.value` and dual values from `constraint.dual_value`, and checks all four conditions:

- (i) **Stationarity:** $\|\nabla_x \mathcal{L}(x^*, \lambda^*)\| \leq \epsilon$
- (ii) **Primal feasibility:** $g_i(x^*) \leq \epsilon$ for all i
- (iii) **Dual feasibility:** $\lambda_i^* \geq 0$ for inequality constraints
- (iv) **Complementary slackness:** $|\lambda_i^* \cdot g_i(x^*)| \leq \epsilon$ for all i

I will also add an optional flag `problem.solve(check_kkt=True, kkt_tol=1e-4)` that runs the check automatically after every solve (default `False` for backward compatibility).

Step 2: Cone-specific optimality tests. CVXPY canonicalizes DCP problems into cone programs. Different cone types use different dual recovery formulas, and the Lagrangian class has never been systematically tested against known solutions for each type. For each major cone, I will construct a small problem with an analytically verifiable optimal solution and known duals, solve it, and assert that `check_kkt()` passes:

- **LP / QP** (nonneg + zero cones): box-constrained least-squares with a vertex solution where KKT conditions can be verified analytically.
- **SOCP** (second-order cone): $\min \|x\|_2$ s.t. $\mathbf{1}^\top x = 1$, with closed-form optimal and duals.
- **SDP** (PSD cone): a small 3×3 instance with a known rank-1 solution and dual matrix satisfying complementarity.
- **Exponential cone:** $\min -\sum_i \log(x_i)$ s.t. $\mathbf{1}^\top x = 1$, with the analytic center as solution.
- **Power cone:** a geometric mean problem with Arithmetic Mean-Geometric Mean inequality–based analytic solution.

Each test will be run across multiple solvers (Clarabel, SCS, MOSEK if available) to confirm solver-agnostic dual recovery.

Step 3: DNLP dual variable support. When CVXPY solves a DNLP problem via IPOPT, the canonicalization path differs fundamentally from the cone path. The DNLP extension transforms the user’s problem into an NLP of the form

$$\min_{(x,t,v,\dots)} \tilde{f}(x,t,v,\dots) \quad \text{s.t.} \quad \tilde{c}(x,t,v,\dots) = 0, \quad \ell \leq (x,t,v,\dots) \leq u$$

where $t, v, \dots$ are auxiliary variables introduced during canonicalization. IPOPT returns multipliers `mult_g` (for equalities) and `mult_x_L`, `mult_x_U` (for bounds) in this canonicalized space, but currently CVXPY discards them—so `constraint.dual_value` is always `None`.

I will implement the inverse dual mapping by storing constraint-to-internal metadata during the forward pass and using it during `invert()`. The mapping depends on the constraint type:

- For a user inequality $g(x) \leq 0$ that became a linking constraint $t - g(x) = 0$ with bound $t \geq 0$: the user-space dual is the multiplier on the linking equality.
- For a user equality $h(x) = 0$ that remained as-is in the NLP: the user-space dual is directly the corresponding `mult_g` entry.
- For nonsmooth reformulations (e.g., $|x| \leq t$ became $-t \leq x \leq t$): the user-space dual aggregates the bound multipliers ($\mu^+ + \mu^-$).

I will validate by solving the path planning and circle packing examples from the DNLP paper [4] and checking that recovered duals satisfy KKT.

Step 4: DNLP KKT verification. For the DCP path, the Lagrangian class computes $\nabla_x \mathcal{L}$ via CVXPY’s atom-level `.grad` and chain rule. For DNLP, I will take a different approach: the DNLP extension already uses SparseDiffPy to provide IPOPT with gradients and Jacobians. Rather than duplicating derivative logic, I will use SparseDiffPy to compute $\nabla_x \mathcal{L}(x^*, \lambda^*)$ directly on the user-space Lagrangian. This is simpler (no need to extend the chain-rule class for each new atom), more general (handles any atom SparseDiffPy supports), and verifies KKT in the same space the user wrote their problem in.

Concretely, the `check_kkt()` method will detect whether the problem was solved with `nlp=True` and switch to the SparseDiffPy code path automatically.

Step 5: DNLP atom coverage. DNLP introduces atoms beyond the standard DCP set. I will write test problems for each category:

Atom	Category	Test Problem	Reference
<code>multiply(x, y)</code>	smooth nonconvex	NMF: $\min \ A - XY\ _F^2$	DNLP §5.4
<code>sin, cos, tan</code>	smooth nonconvex	car control: $x_{k+1} = f(x_k, u_k)$	DNLP §5.7
<code>quad_form(x, Q)*</code>	smooth nonconvex	$\max x^\top A x$ s.t. $\ x\ ^2 = 1$	DNLP README
<code>sigmoid</code>	smooth nonconvex	trimmed logistic regression	DNLP §5.8
<code>norm1</code>	NS-convex	sparse recovery	DNLP §5.6
<code>norm2</code>	NS-convex	circle packing: $\ c_i - c_j\ _2 \geq r$	DNLP §5.2
<code>max</code>	NS-convex	circle packing objective	DNLP §5.2
<code>min</code>	NS-concave	custom test	—
<code>sum_largest</code>	NS-convex	custom test	—

Table 1: DNLP atom test coverage plan. *With indefinite Q . DNLP § references are sections in [4].

Each test will solve via IPOPT, verify primal feasibility, run the DNLP-mode `check_kkt()`, and where possible compare against a known analytical solution or reference solver.

Deliverables

Deliverable	Track	Target Week	hrs/wk
Codebase review report (canonicalization + GSoC 2023 code)	Both	Week 1	~35
Post-solver feasibility check module + solver-tolerance integration	A	Weeks 2–3	~35
Comprehensive feasibility tests (edge cases, all solvers) + warning/override logic	A	Week 4	~35
Integrated & cleaned-up Lagrangian class with <code>check_kkt()</code>	B	Weeks 4–5	~35
<i>~June 20: Robotics research position begins; pace shifts to 15–20 hrs/wk</i>			
Feasibility check user documentation	A	Week 6	~17
<i>Midterm evaluation checkpoint</i>		<i>Week 6</i>	
Cone-specific optimality tests (LP, SOCP, SDP, Exp, Power)	B	Weeks 7–8	~17
DNLP dual variable inverse mapping + KKT verification	B	Weeks 9–10	~17
DNLP atom test coverage	B	Weeks 11–12	~17
Developer documentation on optimality verification	B	Week 13	~17
Final review, cleanup, and PR preparation	Both	Week 14	~17

Table 2: Timeline of deliverables. Weeks 1–5 are front-loaded at ~35 hrs/wk before the robotics research commitment begins; Weeks 6–14 proceed at 15–20 hrs/wk. Estimated total: ~340 hours.

Timeline

Community Bonding Period (Early–Mid May)

- Set up development environment; build CVXPY from source; run existing test suite.
- Read the canonicalization pipeline in depth (`reductions/`, `atoms/`, `cones/`).
- Locate and study Aryaman Jeendgar’s GSoC 2023 Lagrangian prototype code.
- Meet with mentors to align on API design decisions (e.g., where to hook into `solve()`, naming conventions, tolerance defaults).

Coding Period

My university semester ends on **May 20**, giving me full availability until **June 20**, when I begin a robotics research position (32 hrs/wk). To account for this, I adopt a **front-loaded** schedule: the implementation-heavy work is concentrated in Weeks 1–5 (~35 hrs/wk), while testing, documentation, and DNLP extensions—tasks that are more modular and can proceed in shorter sessions—are scheduled for Weeks 6–14 (~15–20 hrs/wk).

Phase 1: Full Availability (Weeks 1–5, ~35 hrs/wk).

Week 1: Codebase Review. Produce a written summary of: (a) the canonicalization chain for cone programs, (b) the DNLP canonicalization chain, (c) the GSoC 2023 **Lagrangian** class—what works, what needs rewriting. Share with mentors for feedback.

Weeks 2–3: Track A — Feasibility Checks. Implement the feasibility verification module: the core `constraint.violation()` loop, solver-tolerance reading for each supported solver, the $\tau = \max(\text{user_tol}, k \times \text{solver_tol})$ logic, and the warning/status-override mechanism. This is the core deliverable that benefits most from long uninterrupted coding sessions.

Week 4: Track A — Edge-Case Tests. Write comprehensive tests covering all the edge cases described above: empty constraints, mixed equality/inequality, scaling issues, infeasible and unbounded problems, complex variables, PSD constraints, and solver-by-solver tolerance verification.

Week 5: Track B — Lagrangian Integration. Port the Lagrangian class into `cvxpy/verification/`, wire it into `Problem.check_kkt()`, and write initial tests on simple LP/QP problems to make sure the basic pipeline works end-to-end.

Phase 2: Reduced Availability (Weeks 6–14, ~15–20 hrs/wk).

Week 6: Feasibility Documentation + Midterm. Write user-facing documentation for feasibility checks: how to enable them, how to interpret warnings, what tolerances to expect. Prepare midterm evaluation materials.

Weeks 7–8: Cone-Specific Optimality Tests. Write and validate optimality tests for all major cone types (LP, SOCP, SDP, exponential, power) with analytically verifiable solutions. Run each test across multiple solvers.

Weeks 9–10: DNLP Dual Variables + KKT Verification. Implement the inverse dual mapping for IPOPT-solved problems (storing constraint metadata during canonicalization, recovering user-space duals in `invert()`). Then build the SparseDiffPy-based DNLP mode for `check_kkt()`. Validate on the DNLP paper’s examples (path planning, circle packing).

Weeks 11–12: DNLP Atom Coverage. Write optimality/feasibility tests for all DNLP-specific atoms (see Table 1): `multiply`, `trig` functions, `sigmoid`, `abs`, `norm1`, `norm2`, etc.

Weeks 13–14: Documentation & Final Polish. Developer documentation on the verification architecture, final code review, CI/linting fixes, and PR preparation. Week 14 serves as buffer for any spillover.

About Me

I am a third-year Electrical and Electronics Engineering student at **Bilkent University**, where I hold a Full Merit Scholarship. My coursework includes signals and systems, linear algebra, probability and statistics, numerical methods, and several programming-intensive courses in Python and C.

In 2025, I was a Visiting Student at **Stanford University**, where I earned an A in EE364A (Convex Optimization). During my time there, I deeply enjoyed participating in weekly problem-solving meetings with my instructor, Daniel Cederberg, and the convex optimization folks. This collaborative environment profoundly solidified my theoretical foundation in optimization.

Building on that foundation, I have been conducting remote research at Stanford since October 2025 focusing directly on CVXPY. My ongoing work targets optimality conditions for Disciplined Nonlinear Programming (DNLP) and feasibility checks via maximum constraint violation—the exact core objectives of this GSoC proposal. In preparation for this project, I have been holding bi-weekly meetings with William Zhang to discuss core concepts, evaluate technical approaches, and carefully define the scope of the deliverables proposed here.

Relevant Skills

- **Mathematical Optimization & CVXPY:** Deep theoretical foundation built through excelling in EE364A (Convex Optimization) at Stanford and IE411 (Non-linear Optimization) at Bilkent. Understanding of duality, KKT conditions, and cone programs (Boyd & Vandenberghe).
- **Python & Scientific Computing:** Proficient in Python, NumPy, SciPy, Pandas, and deep learning frameworks. Extensive hands-on experience utilizing CVXPY and CVX (IE 411) for complex optimization implementations and mathematical modeling.
- **Software:** Comfortable with Git, GitHub workflows, unit testing (pytest/unittest), and CI pipelines. Experience building and open-sourcing algorithms from scratch, such as Safe Swarm Control with Control Barrier Functions (see my webpage/GitHub for further details). Proficient in Python, C, and MATLAB.
- **NLP Solvers:** Familiarity with IPOPT and optimality conditions through ongoing research. Since DNLP canonicalization is not well-documented, I learned its general structure by studying the paper *Code Generation for Embedded Second-Order Cone Programming* with William Zhang, which follows a similar approach.

Open-Source Contributions

I have been actively working on the CVXPY codebase in preparation for this project. Below is a contribution I have developed with a working patch and tests. Some edge cases are still being finalized; I plan to submit a pull request once they are fully validated.

- **Feature: DNLP dual variable recovery** (Issue #3200). When CVXPY solves a DNLP problem via IPOPT, the solver returns Lagrange multipliers in `info['mult_g']`, but the current `IPOPT.invert()` discards them—it always returns `dual_vars={}`, so `constraint.dual_value` is `None` for every DNLP problem. I implemented a three-part fix across two files:
 (1) In `Bounds.get_constraint_bounds()`, I store a metadata list that records, for each constraint, its lowered ID, size, shape, and whether it is an equality:


```
self.constr_meta = []
for constraint in self.problem.constraints:
    if isinstance(constraint, Equality):
        lowered = lower_equality(constraint)
        self.constr_meta.append(
            (lowered.constr_id, constraint.size,
             constraint.shape, True))
    elif isinstance(constraint, Inequality):
        lowered = lower_ineq_to_nonneg(constraint)
        self.constr_meta.append(
            (lowered.constr_id, constraint.size,
             constraint.shape, False))
```

 (2) In `IPOPT.solve_via_data()`, I attach this metadata to the solution dictionary so that `invert()` can access it:


```

    if hasattr(data.get("_bounds", None), "constr_meta"):
        info['_constr_meta'] = data["_bounds"].constr_meta
(3) In IPOPT.invert(), I slice mult_g using the stored metadata, reshape each segment to
match the original constraint shape, and negate inequality duals to align with CVXPY's
sign convention (IPOPT uses  $\mathcal{L} = f + \lambda^T g$ , so active-lower-bound multipliers are negative,
whereas CVXPY's conic solvers return nonneg duals for NonNeg constraints):
    for constr_id, size, shape, is_eq in constr_meta:
        dual_vals = mult_g[offset:offset + size]
        if not is_eq:
            dual_vals = -dual_vals
        dual_vars[constr_id] = np.reshape(
            dual_vals, shape, order='F')
    offset += size

```

The core mapping logic is implemented and verified against 12 existing NLP tests with zero regressions. Additional edge-case tests—including KKT stationarity verification and conic-vs-NLP dual consistency checks—are still being finalized. This work is a direct prerequisite for the optimality-checking deliverables of this proposal (Track B, Step 3). *[PR in preparation; pending final edge-case validation.]*

Time Commitment & External Obligations

My university semester at Bilkent ends on **May 20, 2026**. From May 20 to June 20 I have no external commitments and will dedicate **~35 hours per week** to GSoC.

Starting **June 20**, I will begin a robotics research position that requires approximately 32 hours per week. During this period I will dedicate **15–20 hours per week** to GSoC, primarily during evenings and weekends.

To compensate, I have **front-loaded** the schedule: all implementation-heavy modules (feasibility check core, Lagrangian class integration) are completed before June 20, while modular tasks (tests, documentation, DNLP atom coverage) are spread across the reduced-hours phase. I have also extended the timeline to 14 weeks, using the final two weeks as buffer. The estimated total effort is ~340 hours.

I am fully transparent about this constraint and will discuss pacing adjustments with mentors throughout the program. If needed, I am open to requesting a GSoC end-date extension as allowed by the program rules.

Communication

I am available via GitHub, Slack, and email. I am comfortable with weekly or bi-weekly video calls for progress updates and code reviews, and I will maintain a development log shared with mentors.

AI Disclosure

In accordance with CVXPY's AI disclosure policy, I acknowledge that AI tools were used to assist with structuring and drafting portions of this proposal. All technical content has been reviewed and validated by me personally. During the coding period, any use of AI tools will be disclosed in each pull request as required.

References

- [1] R. J. Murray, “Feature suggestion: post-solver feasibility tolerance checks,” GitHub Issue #434, cvxpy/cvxpy, 2018. <https://github.com/cvxpy/cvxpy/issues/434>
- [2] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. <https://web.stanford.edu/~boyd/cvxbook/>
- [3] A. Jeendgar, “GSoC 2023: CVXPY Optimality Conditions,” Google Summer of Code Archive, 2023. <https://summerofcode.withgoogle.com/archive/2023/projects/a8YyGIP0>
- [4] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, “Disciplined Nonlinear Programming,” December 2025. <https://web.stanford.edu/~boyd/papers/pdf/dnlp.pdf>
- [5] S. Diamond and S. Boyd, “CVXPY: A Python-embedded modeling language for convex optimization,” *Journal of Machine Learning Research*, 17(83):1–5, 2016.
- [6] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, “A rewriting system for convex optimization problems,” *Journal of Control and Decision*, 5(1):42–60, 2018.